Software Requirements Specification

Department Course Timetable Management System | SENG 271

Field	Value
Version	SRS v2 - corrected working revision
Prepared	10.11.2026
Team	Team 19
Members	Eren KARAKAŞ (@ErennKarakass); Semih Mertcan İÇYER (@semihicyer)
Repository	https://github.com/ErennKarakass/Daviser
Status	Ready for team review; no claim of historical GitHub submission.

Revision History

Version	Date	Change
Earlier v2 draft	10.11.2026	Team-provided PDF used as the starting draft. Its historical v1 date and uploads have not been verified.
v2 corrected	10.11.2026	Clarified data formats, room-type derivation, preferences, regeneration, and development status. No earlier Git history is invented.

1. Introduction

Purpose: specify observable requirements for the coordinator, developers and evaluator. Scope: a standalone Java console application for four year groups, file loading, automatic weekly timetables, conflict reporting, manual edits and room-request drafts.

Out of scope: database, web frontend, individual student enrollment/elective clashes, network communication, automatic email sending, advanced optimizers and mandatory automated test frameworks.

References: Term Project Assignment; SENG 271 Term Project Guide; Java Project and GitHub Teamwork Tutorial; instructor Deliverable Templates. These instructor documents control the project scope.

1.3 Definitions

Term	Meaning
Session / block	One course's theory (T) or lab (L) hours grouped into one consecutive interval.
Time slot / interval	A scheduling unit is one hour; the TimeSlot design object can represent an entire block interval.
Year group	Year 1, 2, 3 or 4; all courses in a group share students in this project.
Hard / soft constraint	Hard: reject violating assignments. Soft: prefer satisfaction, otherwise accept with warning.
Special request	Block-specific fixed-time, room, or lab-order instruction.
Unscheduled session	A block left without a valid assignment by this algorithm; this does not prove no valid timetable exists.

2. Overall Description

User: the Department Timetable Coordinator knows curriculum data and scheduling restrictions; no Java programming knowledge is assumed. The coordinator loads data, generates and reviews schedules, resolves failures, and saves outputs.

Environment: JDK 21 is the proposed common baseline; JDK 25 also works if both members agree. Console application; no external Java libraries. Start from IntelliJ using Main, or use the README compile/run commands. Team machines are not yet verified.

2.4 Assumptions and Decisions

Monday-Friday, 09:00-17:00; 12:00-13:00 is a normal slot. Day values 1-5 denote Monday-Friday. Intervals are half-open: a block ending at 12:00 does not clash with one starting at 12:00. A theory and lab block may occur on the same day if theory finishes first. Zero-hour blocks are omitted; both hour counts cannot be zero. Each individual block is 1-8 hours, never split. Room type derives from T/L, rather than an extra course-file column. Sample formats and datasets require instructor approval.

3. Functional Requirements (1/2)

ID	The system shall ...	Priority / source
FR-01	Read course data from data/courses.csv.	Must / §3.7
FR-02	Skip malformed records with file name, line number and reason, without crashing.	Must / §3.7
FR-03	Read instructor identity, unavailable intervals and preferred intervals from data/instructors.txt.	Must / §3.2, 3.7
FR-04	Read room type, capacity and unavailable intervals from data/rooms.csv.	Must / §3.4, 3.7
FR-05	Read block-specific requests from data/constraints.txt and distinguish mandatory from preferred requests.	Must / §3.6, 3.7
FR-06	Generate a weekly timetable automatically with no hard-constraint violations.	Must / §3.8
FR-07	Process fixed-time mandatory blocks before flexible blocks, without moving failed fixed blocks to other times.	Must / §3.8
FR-08	Try alternative allowed rooms and time slots before declaring a session unassigned.	Must / §3.8
FR-09	Detect instructor, room and year-group clashes and all other HC-01 to HC-09 failures.	Must / §3.5, 3.8
FR-10	Keep an unplaceable block unassigned and record an understandable failure reason.	Must / §3.8
FR-11	Display each year group's weekly timetable with course code/title, instructor, day, interval and room.	Must / §3.9, 3.11
FR-12	Display a list of all scheduled sessions.	Must / §3.9
FR-13	Display unassigned sessions with NO ROOM or CONSTRAINT classification and reasons.	Must / §3.9

3. Functional Requirements (2/2)

ID	The system shall ...	Priority / source
FR-14	Show a warning when an accepted assignment violates a soft preference.	Must / §3.5
FR-15	Allow the coordinator to move an assignment to a new day, interval and room.	Must / §3.9
FR-16	Reject invalid manual moves using the same hard checks, keeping the original valid assignment.	Must / §3.9
FR-17	Allow assignment removal, leaving the session unassigned.	Must / §3.9
FR-18	Distinguish room shortage from instructor, year-group or mandatory-time failures.	Must / §3.10
FR-19	Display a formal room-request email draft containing the course, students, required room type and a suggested time when available.	Must / §3.10
FR-20	Save the email draft to a text file chosen by the coordinator, without sending email.	Must / §3.10
FR-21	Offer text-file export of timetables and scheduling reports.	Should / §3.11
FR-22	Consider theory-before-lab, instructor preferences and year-group gaps when selecting candidates.	Must / §3.5
FR-23	Allow regeneration from the loaded data, clearing prior assignments and reports.	Must / §3.9
FR-24	Report a missing required file and leave the previous complete dataset unchanged.	Must / §3.7
FR-25	Maintain course, instructor, room, session and request objects for the loaded dataset.	Must / §3.1-3.4

4.1 Hard Constraints

ID / rule	Required behavior and data	Violation message
HC-01 Instructor conflict	The same instructor cannot teach overlapping blocks. Data: Course.instructor; assigned intervals	Instructor already busy
HC-02 Room conflict	A room cannot host overlapping blocks. Data: Assigned rooms and intervals	Room already occupied
HC-03 Year-group conflict	All courses of the same year group are assumed to share students and cannot overlap. Data: Course.year; assigned intervals	Year group already busy
HC-04 Working hours	Each entire block stays Mon-Fri, 09:00-17:00. Data: Day, start, duration	Block outside working hours
HC-05 Capacity	Room capacity is at least course student count. Data: Room.capacity; Course.students	Insufficient room capacity
HC-06 Room type	T blocks use Classroom; L blocks use Laboratory. Data: Session type; Room.type	Wrong room type
HC-07 Instructor availability	No part of a block overlaps instructor unavailability. Data: Instructor.unavailableSlots	Instructor unavailable
HC-08 Fixed time	A mandatory fixed block keeps its exact required day, start and duration. Data: FIXED_TIME request; candidate interval	Mandatory fixed interval violated
HC-09 Room availability	No part of a block overlaps room unavailability. Data: Room.unavailableSlots	Room unavailable

Mandatory specific-room requests restrict candidate rooms even though they are additional to the nine named rules. Conflicting mandatory requirements are reported; no invalid assignment is created.

4.2 Soft Constraints

ID	Consideration	Warning / handling
SC-01	Theory before lab: Prefer a lab interval after the end of its theory block; same day is allowed. LAB_BEFORE_THEORY permission suppresses this warning.	Lab precedes theory, or theory remains unassigned.
SC-02	Instructor preferences: Try actual declared preferred intervals first; do not assume all instructors prefer mornings.	Assigned outside declared preferences.
SC-03	Avoid gaps: Prefer candidates producing fewer empty hours between a year group's first and last class on that day.	Assignment leaves a year-group gap.
SC-04	Consecutive hours: Every teaching block is inherently consecutive; nonconsecutive fragments are never candidates.	No split-block acceptance; invalid blocks are rejected.

4.3 Special Requests

Type / example	Class	Handling
FIXED_TIME: SENG301-T; 3@13-16	Mandatory or preferred	Mandatory: only this interval. Preferred: try first, otherwise warn.
SPECIFIC_ROOM: SENG401-L; B202	Mandatory or preferred	Mandatory: only this room. Preferred: try first, otherwise warn.
Instructor day off: 5@09-17	Mandatory availability	Stored in instructors.txt and checked under HC-07.
LAB_BEFORE_THEORY: SENG302-L; ALLOW	Preferred / exception permission	Allows that lab to precede its theory; it does not force lab-first placement.

5. Input Data Requirements

UTF-8 text; trim surrounding whitespace. Ignore blank lines and lines starting with #. No header row is required; examples of column names belong in # comments. CSV uses commas; TXT uses semicolons. Names/titles must not contain delimiters, quotes or embedded newlines (simple parser scope). An empty interval list is represented by -. All files are required, but constraints may contain comments only.

File under data/	Ordered fields	Example line
courses.csv	CSV; code,title,year,instructorId,theoryHours,lab Hours,students	SENG201,Object-Oriented Programming,2,I02,3,2,55
instructors.txt	TXT; id;name;unavailableSlots;preferredSlots	I02;Dr. Demir;1@09-12|5@09-17;2@09-12
rooms.csv	CSV; id,type,capacity,unavailableSlots	B202,Laboratory,50,3@09-11
constraints.txt	TXT; sessionId;requestType;value;mandatory	SENG201-T;FIXED_TIME;2@09-12;true

Intervals: day@HH-HH, e.g. 1@09-12; multiple intervals separated by |. Day is 1-5, start 09-16, end 10-17, end greater than start. Preferred intervals are optional. A block satisfies a preferred interval only when fully contained.

Validation: nonempty unique IDs; year 1-4; integer hours 0-8, at least one block; positive integer students and capacity; room type exactly Classroom or Laboratory. Instructor references must exist. Requests reference an existing generated session, use a known type, and have boolean true/false. FIXED_TIME length must equal block duration; SPECIFIC_ROOM references an existing room; LAB_BEFORE_THEORY uses ALLOW and false. Contradictory valid mandatory requests are retained and reported as scheduling conflicts.

Load order: instructors, rooms, courses, generated sessions, requests. Duplicate IDs after the first valid record are skipped. Missing files abort the whole load without replacing a previous complete dataset; invalid rows are skipped with file/line/reason. Unknown references are rejected. No scheduling starts without a successfully loaded dataset.

6. Output Requirements

Output	Contents and destination
Year timetable	Console: full course, teacher, day/time and room information. Optional output/timetable_year1.txt through year4.txt.
Scheduled list	Console and optional output/scheduled_sessions.txt.
Unscheduled and warnings	Session ID, NO ROOM/CONSTRAINT and reason; soft/input warnings; optional output/unscheduled_report.txt.
Room-request draft	Formal greeting, course/title, students, room type and suggested time (if available), closing. Display and save output/classroom_request_SESSION.txt. Never send email.

7. Non-Functional Requirements

ID	Requirement
NFR-01	Do not terminate unexpectedly on common invalid input or file errors; provide an understandable message.
NFR-02	Reject nonnumeric and out-of-range menu choices and show the menu again; allow option 0 to exit.
NFR-03	Target generation within 3 seconds for a documented sample of at most 20 courses and 8 rooms, on a recorded JDK 21/25 desktop test environment. Measure in a later version.
NFR-04	Keep class names, declared attributes and method signatures consistent across source and current SDD.

8. Changes from the Supplied v2 Draft

FR-14 warning behavior is required; FR-21 export remains optional per the assignment. Added FR-22 to FR-25 for preference consideration, regeneration, missing-file behavior and object management. Added precise examples, parsing rules, session-level requests and room-type derivation. Replaced unsupported historical revision claims with a transparent current revision record. Documented an explicit performance test scope.

AI-Use Declaration

Tool	Used for	Verification / remaining review
ChatGPT (Codex)	Requirements revision, design and diagram generation, Java skeleton, documentation and build checks.	Assistant checked source/diagram alignment and compiled/exercised the skeleton in its environment. Both team members must still review, run and explain the result; their approval is not claimed.